

บทที่ 2

ทฤษฎีคอมโพเนนต์

2.1 การพัฒนาเชิงคอมโพเนนต์

คอมโพเนนต์ คือ ส่วนย่อยของระบบที่ไม่ขึ้นอยู่กับส่วนอื่นๆ และถูกซ่อนรายละเอียดไว้ภายใน ซึ่งหน้าที่ของคอมโพเนนต์จะถูกกำหนดไว้อย่างเหมาะสมเท่าที่ไคลเอ็นต์ต้องการใช้งานเท่านั้น ส่วนการจำกัดหน้าที่บางอย่างของคอมโพเนนต์นั้นจะกระทำโดยสถาปัตยกรรม (Architecture) หรือ คอนเทนเนอร์ (container) ที่คอมโพเนนต์นั้นดีพลอย (deploy) อยู่

คอมโพเนนต์อาจพัฒนา ทดสอบ และดีพลอยในสภาพแวดล้อมที่แยกจากส่วนอื่นๆ ของระบบ ถึงแม้ว่าในความเป็นจริงนั้น คอมโพเนนต์จะถูกใช้งานร่วมกับคอมโพเนนต์อื่นๆ ก็ตาม แต่การทำแบบนี้มีความสำคัญต่อการทดสอบและคุณภาพของคอมโพเนนต์

เราอาจพิจารณาว่าส่วนใดๆ ของระบบซอฟต์แวร์เป็นคอมโพเนนต์ได้ เช่น ออบเจกต์ เมธอด เพราะระบบถูกสร้างขึ้นจากการประกอบของส่วนต่างๆ นี้ แต่การพิจารณาแบบนี้ไม่เหมาะกับการพัฒนาซอฟต์แวร์เชิงคอมโพเนนต์ เพราะหมายความว่าผู้พัฒนาจะต้องเกี่ยวข้องกับคอมโพเนนต์เสมอ

เพื่อความหมายที่สมบูรณ์ของคอมโพเนนต์จะต้องพูดถึงอินเทอร์เฟซ เพราะไคลเอ็นต์จะใช้อินเทอร์เฟซในการทำงานกับคอมโพเนนต์

การพัฒนาเชิงคอมโพเนนต์นั้น มีความหมายกับคนแต่ละกลุ่มแตกต่างกันไปตามความต้องการ ซึ่งมีลักษณะเด่นบางอย่างที่นิยามความเป็นระบบคอมโพเนนต์ดังต่อไปนี้

1. คอมโพเนนต์มีลักษณะของออบเจกต์ คือ การซ่อนรายละเอียด (Encapsulated), โพลีมอร์ฟิก(polymorphic), การกำหนดหน้า, การกำหนดอินเทอร์เฟซ
2. คอมโพเนนต์ออกแบบภายใต้เฟรมเวิร์ก (Framework) ซึ่งได้สร้างข้อจำกัดบางอย่างไว้ เช่น ต้องไม่มีหลายเทรด, ไม่มีการติดต่อกับภายนอกโดยไม่ผ่านบริการของเฟรมเวิร์ก
3. คอมโพเนนต์สามารถอยู่ได้โดยไม่พึ่งพาคอมโพเนนต์อื่นๆ ยกเว้นคอมโพเนนต์ของเฟรมเวิร์กที่คอมโพเนนต์ดังกล่าวใช้อยู่
4. ทุกคอมโพเนนต์มีอินเทอร์เฟซสามัญที่แน่นอน (Fix and common) และอินเทอร์เฟซนี้เปลี่ยนแปลงไม่ได้
5. คอมโพเนนต์สามารถอธิบายตนเองได้ โดยอินเทอร์เฟซของคอมโพเนนต์จะต้องมีข้อมูลมากพอที่สามารถทำให้ไคลเอ็นต์สามารถเข้าใจวิธีใช้คอมโพเนนต์นั้นได้

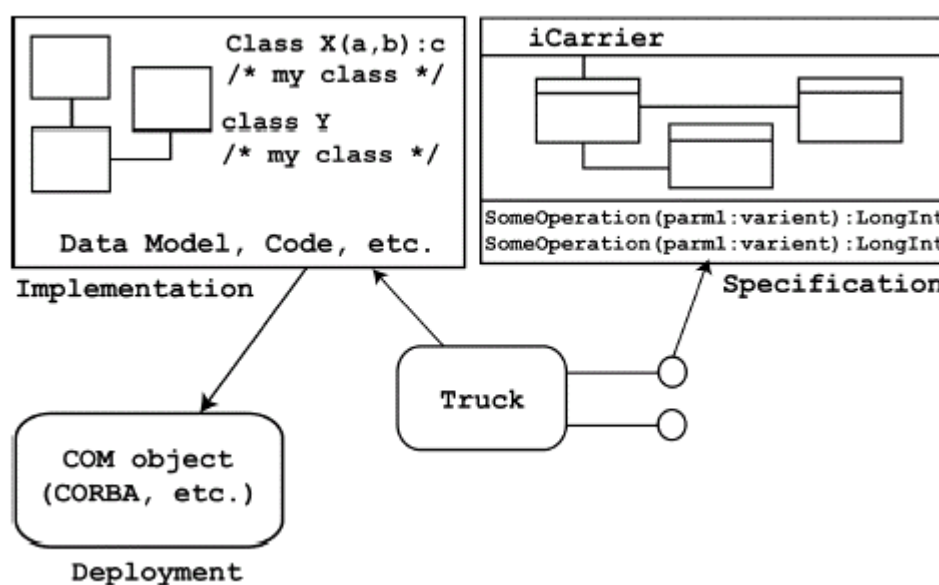
2.1.1 เฟรมเวิร์ก คือ สภาพแวดล้อมที่มีลักษณะดังนี้

1. สร้างอินสแตนซ์ของคอมโพเนนต์คอนรันไทม์
2. ทำให้คอมโพเนนต์ค้นพบคอมโพเนนต์อื่นๆ ได้
3. ทำให้คอมโพเนนต์ต่างๆ สามารถติดต่อกันได้
4. จัดหาบริการสามัญ เช่น เพอร์ซิสเทนซ์ (persistence), ทรานส์แอ็กชัน (transaction), ความไม่ขึ้นกับสถานที่, การรักษาความปลอดภัย, มอนิเตอร์ริง (monitoring)

2.1.2 ส่วนประกอบของคอมโพเนนต์

คอมโพเนนต์จะประกอบด้วยส่วนสำคัญ 3 ส่วนดังนี้

- อินเทอร์เฟซ (interface) - คอมโพเนนต์จะถูกเรียกใช้งานผ่านทางอินเทอร์เฟซ
- อิมพลีเมนต์เตชัน (implementation) - เป็นโค้ดที่กำหนดการทำงานของคอมโพเนนต์
- ดีพลอยเมนต์ (deployment) - เป็นเอ็กซิกิวต์ไฟล์จะใช้ในการทำให้คอมโพเนนต์ทำงานได้ ทำหน้าที่จัดหารันไทม์เอ็นไวรอนเมนต์ในการควบคุมการทำงานของคอมโพเนนต์และจัดหาเซอร์วิสที่จำเป็น

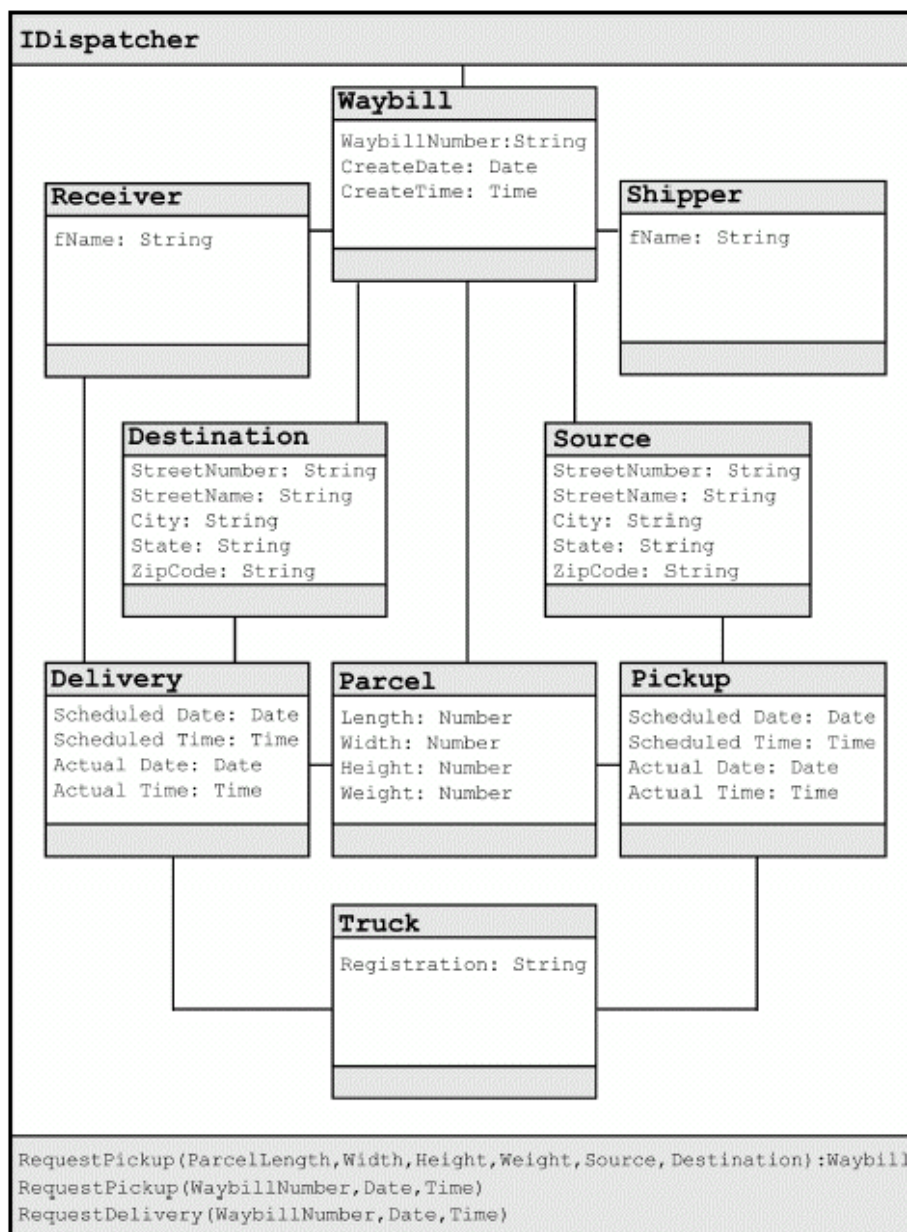


รูปที่ 2-1 คอมโพเนนต์และส่วนประกอบต่างๆ ของมัน

2.1.3 คุณสมบัติของคอมโพเนนต์

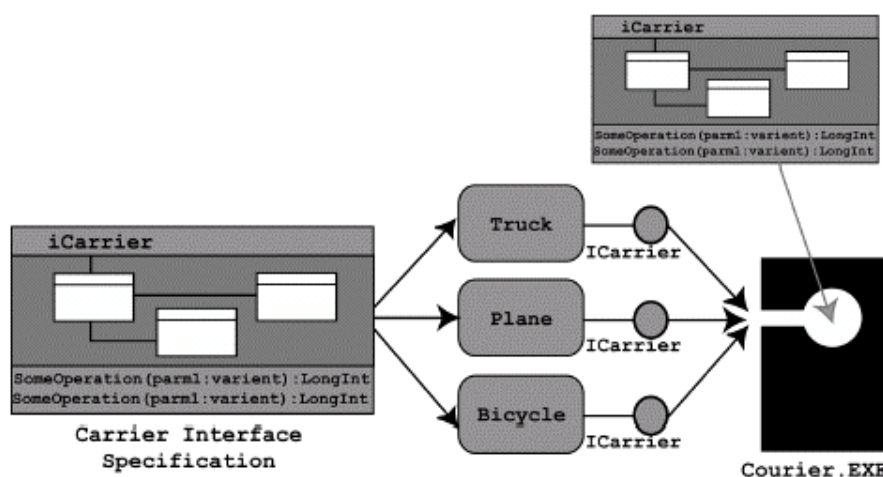
- **Encapsulated** - เป็นกระบวนการในการซ่อนโค้ดหรืออิมพลีเมนต์เตชันของคอมโพเนนต์ ผู้ใช้จะรู้เพียง อินเทอร์เฟซและจะใช้งานผ่านมัน โดยไม่จำเป็นต้องรู้ถึงอิมพลีเมนต์เตชัน ทำให้การเปลี่ยนแปลงอิมพลีเมนต์เตชันไม่มีผลกระทบต่อผู้ใช้

- **Descriptive** - เนื่องจากคอมโพเนนต์จะต้องติดต่อผ่านอินเทอร์เฟซเท่านั้น มันจะต้องมีอินฟอร์เมชันของมันเองที่ผู้ใช้สามารถจะเข้าใจได้ โดยอินฟอร์เมชันนั้นจะต้องอธิบายถึงอินเทอร์เฟซ , อิมพลีเมนต์เดชัน และดีพลอยเมนต์



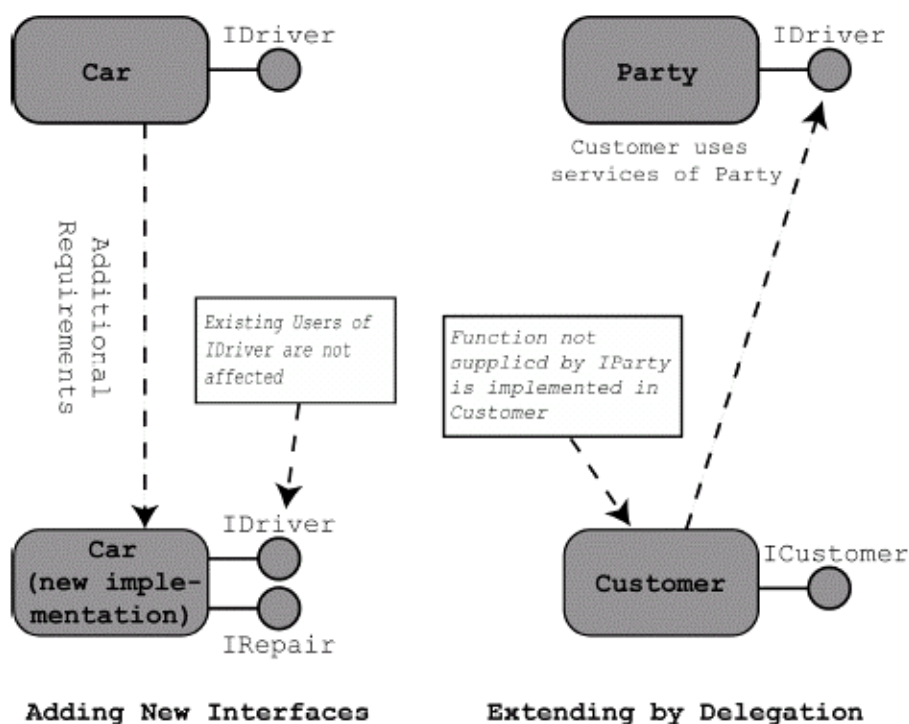
รูปที่ 2-2 ตัวอย่างของอินเทอร์เฟซ

- **Replaceable** - คอมโพเนนต์มีความสามารถในการเปลี่ยนคอมโพเนนต์หนึ่งกับคอมโพเนนต์ใด ๆ ที่มีอินเทอร์เฟซเหมือนกัน



รูปที่ 2-3 คอมโพเนนต์และการแทนที่

- **Extensible** - สามารถที่จะเพิ่มความสามารถได้ มี 2 วิธีคือ การเพิ่มอินเทอร์เฟซและมอบหมายหน้าที่ (Delegating Responsibility) การสร้างคอมโพเนนต์ใหม่สามารถมอบหมายหน้าที่ให้กับบริการที่มีอยู่ในคอมโพเนนต์ที่มีอยู่แล้ว



รูปที่ 2-4 การเพิ่มเติมคอมโพเนนต์

2.2 งานที่เหมาะสมกับการพัฒนาเชิง คอมโพเนนต์

การพัฒนาเชิงคอมโพเนนต์ เป็นวิธีการพัฒนาคอมโพเนนต์โดยใช้คอมโพเนนต์เฟรมเวิร์ก ซึ่งการคงหน้าที่ของคอมโพเนนต์ไว้อย่างถูกต้อง (Well defined responsibility) กระทำได้โดยให้คอมโพเนนต์เชิงธุรกิจ หลีกเลี่ยงโค้ดที่ไม่สามารถทำงานกับอินเทอร์เฟซที่แน่นอนได้

เฟรมเวิร์กคอมโพเนนต์จะซ่อนบริการต่างๆ ของเอ็นไวรอนเมนต์ ซึ่งบริการต่างๆ เหล่านี้เป็นมาตรฐานแล้ว เช่น message oriented middle-ware, Transaction Monitors, การรักษาความปลอดภัย และพิสูจน์สิทธิ์

2.3 ประโยชน์ของการพัฒนาเชิงคอมโพเนนต์

การพัฒนาเชิงคอมโพเนนต์มีประโยชน์อย่างมากด้านเกี่ยวกับเทคนิค และประโยชน์เหล่านี้ยังนำไปสู่ประโยชน์ด้านธุรกิจด้วย

ประโยชน์ด้านเทคนิค

1. จัดการความซับซ้อนได้ดีขึ้น ทำให้คุณภาพของการแก้ปัญหาดีขึ้น
2. งานซับซ้อนที่ไม่เกี่ยวกับฟังก์ชันธุรกิจ ถูกรวมไว้ในเฟรมเวิร์ก
3. การออกแบบ, อิมพลีเมนต์, ทดสอบ ส่วนต่างๆ อย่างอิสระ ทำให้สามารถพัฒนาส่วนต่างๆ ได้พร้อมกัน
4. การไม่ขึ้นต่อกันของคอมโพเนนต์ลดผลกระทบของการเปลี่ยนแปลง requirement ไม่ให้กระจายไปทั้งระบบ

ประโยชน์ด้านธุรกิจ

1. ผลิตรักษะที่มีคุณภาพมากขึ้น
2. เร่งเวลาออกสู่ตลาด
3. ใช้ทรัพยากรบุคคลได้คุ้มค่าขึ้น
4. ลดค่าใช้จ่าย
5. เพิ่มปริมาณการนำกลับมาใช้ ในโครงการต่อไป

2.4 แนวโน้มของระบบในอนาคต

- ความต้องการระบบที่ซับซ้อนยิ่งขึ้น
- ความต้องการให้ระบบติดต่อกันจากธุรกิจสู่ธุรกิจ (B2B)
- ความต้องการให้ระบบ โดยด้านเครือข่าย ดีขึ้น

คอมโพเนนต์สามารถเติมเต็มความต้องการเหล่านี้ได้ เพราะระบบสามารถสร้างจากระบบย่อยที่อิสระต่อกัน ทำให้ระบบรวมมีความแข็งแกร่งขึ้น เพราะชิ้นส่วนย่อยหนึ่งไม่ยึดติดกับความยุ่งยากภายในชิ้นส่วนย่อยอื่น